🚀 Collections Framework Masterclass

Chapter 21 — ConcurrentHashMap — Complete Deep Dive

> **Interview Importance:** ⭐⭐⭐⭐⭐

`ConcurrentHashMap` is one of the **most important Java collections for senior-level interviews**, especially for questions involving:

- Multithreading
- Concurrent access
- Thread safety
- Atomic operations
- High-throughput applications
- Caching
- Counters
- Java 8+ concurrency

The central idea:

> **ConcurrentHashMap allows multiple threads to safely access and update a Map concurrently without locking the entire Map for every operation.**

Think:

```text id="c1v7m4"
HashMap ↓
Fast ↓
Not thread-safe

ConcurrentHashMap ↓
Fast + concurrent ↓
Thread-safe
```

```
---

# 1. What Is ConcurrentHashMap?

Package:

```java id="x8m3q7"
java.util.concurrent.ConcurrentHashMap
```

It implements:

```text id="r5k2p9"
ConcurrentMap<K,V> ↓
Map<K,V>
```
```

````
Hierarchy:

```text id="n7x4m1"
Map
 ↓
ConcurrentMap
 ↓
ConcurrentHashMap
````

Example:

```java id="p3q8v2"
Map<String, Integer> map = new ConcurrentHashMap<>();
```

````
---

# 2. Why Do We Need ConcurrentHashMap?

Consider:

```java id="m8r2k5"
HashMap<String, Integer> map =
    new HashMap<>();
````

Now multiple threads perform:

```text id="j4x7p1"
Thread 1 → put()
Thread 2 → get()
Thread 3 → put()
Thread 4 → remove()
```

````
HashMap doesn't provide the necessary thread-safety guarantees for concurrent
mutation.

You could synchronize:

```java id="q6m9v3"
synchronized (map) {
    map.put(key, value);
}
````

But now:

```text id="a2r8k4"
Thread 1 ↓ locks entire map
↓
Thread 2 waits
Thread 3 waits
Thread 4 waits
```
```

```
This limits concurrency.

ConcurrentHashMap is designed to allow much more concurrent access.

---

# 3. Main Goal

ConcurrentHashMap tries to achieve:

```text id="v5k3m8"
Thread safety
      +
High concurrency
      +
Good performance
```

rather than:

```text id="n8q2r6"
Thread safety + One giant lock
```

```
---

# 4. HashMap vs ConcurrentHashMap

| Feature | HashMap | ConcurrentHashMap |
|---|---|---|
| Thread-safe | ❌ | ✅ |
| Concurrent reads | Not guaranteed | ✅ |
| Concurrent updates | ❌ | ✅ |
| Null key | One allowed | ❌ |
| Null value | Allowed | ❌ |
| Ordering | None guaranteed | None guaranteed |
| Basic operations | O(1) average | O(1) average |
| Atomic Map operations | Limited | ✅ |
| Intended for concurrency | ❌ | ✅ |

---

# 5. Why Doesn't ConcurrentHashMap Allow Null?

This is a **very common interview question**.
```
```

```java id="f7m3q9"
map.put(null, value);
```

throws:

```text id="x2k8p4"
NullPointerException
```

Likewise:

```java id="j5r9m1"
map.put(key, null);
```

is not allowed.

---

6. Why Are Nulls Prohibited?

The major reason is ambiguity in concurrent operations.

Suppose:

```java id="q8m3x5"
map.get(key)
```

returns:

```text id="v4k7p2"
null
```

Does that mean:

```text id="a6n2r9"
1. key doesn't exist
```

or:

```text id="c5x8m3"
2. key exists with null value
```

For ConcurrentHashMap, eliminating null values removes this ambiguity.

This is particularly important for atomic/concurrent APIs.

7. Example

```java id="m2q7v4" ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

map.put("A", 10);
```

```
Valid.

But:

```java id="r8x3k6"
map.put("B", null);
```

throws:

```text id="p5m9q2" NullPointerException
```

```
---

# 8. Java 7 vs Java 8+ Architecture

This is extremely important for interviews.

### Java 7

ConcurrentHashMap used:

```text id="x4k8m2"
Segments
```

Conceptually:

```text id="v7q3n9" ConcurrentHashMap ├── Segment 1 ├── Segment 2 ├── Segment 3 └── Segment 4
```

```
Each segment acted somewhat like an independently lockable portion.

---

### Java 8+
```

```
The segmented architecture was removed.

The implementation uses:

```text id="r5m2x8"
CAS
+
volatile operations
+
bin-level synchronization
+
tree bins for heavy collisions
```

This is the architecture you should focus on for modern Java interviews.

---

9. Java 7 Segmentation

Conceptually:

```text id="k3p7v1" Map | +-- Segment 0 | +-- Segment 1 | +-- Segment 2 | +-- Segment 3

```
A thread modifying Segment 0 doesn't necessarily block another thread modifying
Segment 3.

This allowed concurrency between segments.

---

10. Why Was Segmentation Removed?

Java 8 introduced a more fine-grained design.

Instead of:

```text id="n8x4m2"
Map
 ↓
large segment locks
```

the implementation can synchronize much smaller portions of the table when necessary.

Conceptually:
```

```text id="q5r9k3" Bucket 0 Bucket 1 Bucket 2 Bucket 3 Bucket 4

```
Only the relevant bin may need locking for certain updates.

This provides better concurrency and flexibility.

---

11. Java 8+ Internal Structure

At a high level:

```text id="w3m8q1"
ConcurrentHashMap
        ↓
      table[]
        ↓
      bins
        ↓
  ┌─────┴──────┐
  ↓            ↓
Node chain   TreeBin
```

A bin can contain:

```text id="j7x2m5" Node → Node → Node

```
or, under heavy collisions:

```text id="p4k8r2"
TreeBin
   ↓
Red-Black Tree
```

---

12. CAS

CAS means:

Compare-And-Swap

Conceptually:

```text id="c8m2q5" Expected value ↓ Compare ↓ If unchanged ↓ Replace atomically
```

````
For example:

```text id="r6x9k1"
current = null
expected = null
newNode = X

CAS(current, null, X)
````

If another thread changed the location first:

```text id="n4p7m3" CAS fails
```

````
and the operation can retry or take another path.

---

# 13. Why CAS Is Useful

CAS allows certain operations to happen without taking a traditional lock.

Conceptually:

```text id="q2m8x4"
Thread A
   ↓
CAS
   ↓
success

Thread B
   ↓
CAS
   ↓
failure
   ↓
retry
````

This is one of the foundations of high-concurrency Java data structures.

---

14. Volatile Reads

ConcurrentHashMap also relies heavily on memory-visibility guarantees.

Conceptually:

```text id="x5k9m2" Thread A writes ↓ memory visibility ↓ Thread B reads

```
Volatile semantics help ensure threads see appropriate updates.

You don't need to memorize every internal memory-barrier detail for most
interviews.

The key terms are:

```text id="j8r3p6"
CAS
volatile
synchronization
```

---

15. Does ConcurrentHashMap Lock the Entire Map?

No.

This is one of the biggest differences from:

```java id="v4m8q2" Collections.synchronizedMap(...)

```
A synchronized Map typically uses a single synchronization mechanism around
operations.

ConcurrentHashMap uses a much more fine-grained approach.

Conceptually:

```text id="a7x3n9"
Thread A → Bin 1
```

```
Thread B → Bin 8
Thread C → Bin 15
```

can make progress concurrently where the implementation permits.

---

16. Important Clarification

Don't say:

> "ConcurrentHashMap never locks."

That's incorrect.

Modern ConcurrentHashMap can use synchronization for certain updates, particularly when modifying an occupied bin.

The better statement is:

> **It avoids a single global lock and uses fine-grained synchronization combined with CAS and volatile operations.**

This is a much stronger interview answer.

---

17. `get()` Operation

Conceptually:

```java id="m5q8r2" map.get(key);
```

does:

```text id="z3x7p1"
key
 ↓
hash
 ↓
table index
 ↓
bin
 ↓
find matching key
```

```
 ↓
return value
```

Reads are designed to be highly concurrent.

---

18. Are Reads Locked?

Normally, simple reads such as:

```java id="f8m3q7" get() containsKey()

```
do not require locking the entire Map.

This is a major reason ConcurrentHashMap scales much better than a single-lock
approach for read-heavy workloads.

---

19. `put()` Operation

Conceptually:

```text id="r2x8m4"
put(key,value)
       ↓
calculate hash
       ↓
find bin
       ↓
empty?
 ┌─────┴─────┐
 ↓           ↓
yes          no
 ↓           ↓
CAS       synchronize bin
 ↓           ↓
insert     update/insert
```

This is a simplified mental model, not the exact source-code algorithm.

---

20. Empty Bin + CAS

If the target bin is empty, the implementation can often attempt to insert using CAS.

Conceptually:

```text id="n6k3p9" empty bin ↓ CAS(null, newNode) ↓ success

```
No conventional lock is required for that insertion path.

---

21. Occupied Bin

If the bin already contains nodes:

```text id="p4m7x2"
bin
 ↓
Node → Node → Node
```

the implementation may synchronize on an appropriate node/bin structure while modifying that bin.

This prevents conflicting updates to that portion of the table.

---

22. Why Fine-Grained Locking Helps

Suppose:

```text id="q8x3m5" Thread A → Bin 2 Thread B → Bin 9

```
They may proceed concurrently.

Contrast this with:

```text id="r7m2k9"
synchronizedMap
       ↓
one lock
```

```
       ↓
only one thread at a time
```

This is the key scalability advantage.

---

23. Hash Collisions

ConcurrentHashMap still has to deal with hash collisions.

Example:

```text id="w5p8x1"
Key A → bucket 5
Key B → bucket 5
Key C → bucket 5
```

```
Conceptually:

```text id="n3k7m4"
Bucket 5
   ↓
A → B → C
```

If collisions become heavy enough, tree-based structures can be used.

---

24. Tree Bins

Similar to modern HashMap, ConcurrentHashMap can use tree bins for heavy collisions.

Conceptually:

```text id="x8m2q6"
Many collisions
↓
TreeBin
↓
Red-Black Tree
```

```
This prevents a heavily contended/collided bin from degrading into a long linear
chain.

---

# 25. Treeification Threshold

Modern Java implementations use thresholds related to HashMap's collision-
management strategy.
```
```

````
You may encounter:

```text id="k4r9p2"
TREEIFY_THRESHOLD = 8
````

and:

```text id="m7x3q5" MIN_TREEIFY_CAPACITY = 64

````
These are implementation details and should not be treated as universal API
guarantees.

For interviews:

> Know that sufficiently collision-heavy bins can become tree-based.

---

26. Resize

ConcurrentHashMap also needs resizing when the table becomes sufficiently full.

Conceptually:

```text id="p2m8v4"
small table
   ↓
threshold exceeded
   ↓
resize
   ↓
larger table
````

But resizing is designed to happen concurrently.

---

27. Concurrent Resizing

One impressive feature is that multiple threads can participate in resizing.

Conceptually:
````

```text id="r5x9k2" Thread A ──┐  Thread B ──┼──→ help transfer bins Thread C ──┘
```

````
This is different from thinking:

> "One thread locks the entire Map and moves everything."

Modern ConcurrentHashMap has mechanisms for cooperative resizing.

---

# 28. `size()`

A common misconception is:

> "`size()` always takes one global lock."

Not necessarily.

ConcurrentHashMap maintains internal counting mechanisms designed for
concurrency.

The exact implementation uses counters/cells and may aggregate counts.

For interview purposes:

> Its size accounting is designed to work efficiently under concurrent updates.

---

# 29. Weakly Consistent Iterators

This is a major interview topic.

ConcurrentHashMap iterators are:

> **Weakly consistent**

They do not behave like ordinary fail-fast iterators.

---

# 30. What Does Weakly Consistent Mean?

Suppose:

```java id="m8q3r5"
for (String key : map.keySet()) {
````
````

```
    ...
}
```

while another thread modifies the Map.

The iterator:

- Does not throw `ConcurrentModificationException` merely because concurrent updates occur.
- May reflect some updates made after iteration begins.
- Is not guaranteed to provide a snapshot of the Map at one exact moment.

Think:

```text id="x4k7p2"
Iteration ↓
sees a moving Map ↓
safe from fail-fast CME ↓
not a snapshot
```

```
---

# 31. Fail-Fast vs Weakly Consistent

### ArrayList / HashMap

Typical iterators are:

```text id="j5m8r1"
fail-fast
```
```

Concurrent modification may result in:

```text id="v7x2q9"
ConcurrentModificationException
```

```
### ConcurrentHashMap

Iterators are:

```text id="p3k6m4"
weakly consistent
```
```

They don't fail-fast merely because another thread modifies the Map.

---

32. Is ConcurrentHashMap Iterator Fail-Safe?

You will often hear:

> "ConcurrentHashMap has fail-safe iterators."

This terminology is common in interview discussions, but **"weakly consistent" is the more precise JDK terminology**.

Use:

> **Weakly consistent iterator**

in a senior-level interview.

---

33. Example

Thread 1:

```java id="n7m4x8" for (Integer key : map.keySet()) { System.out.println(key); }
```

```
Thread 2:

```java id="q5r8p2"
map.put(100, "X");
```

The iterator does not simply fail because of that concurrent modification.

But you should not assume it must see:

```text id="c3x9m6" 100
```

```
either.

The iteration is not a snapshot.

---

# 34. `putIfAbsent()`

This is one of the most useful methods.
```
```

````
Naive approach:

```java id="j8m3q6"
if (!map.containsKey(key)) {
    map.put(key, value);
}
````

This is **not atomic**.

Two threads can do:

```text id="r4x7p2" Thread A → containsKey → false Thread B → containsKey → false

Thread A → put Thread B → put

````
Race condition.

---

35. Correct Approach

Use:

```java id="x5m9k3"
map.putIfAbsent(key, value);
````

This provides an atomic compound operation.

Conceptually:

```text id="p7q2r8" if absent + put

````
happens as one Map operation.

---

36. `computeIfAbsent()`

Another very important method:

```java id="m4x8p1"
map.computeIfAbsent(
    key,
````
````

```
    k -> createValue(k)
);
```

Meaning:

> If the key isn't present, compute and associate the value.

This is especially useful for:

- Caches
- Grouping
- Per-key state
- Lazy initialization

---

37. Example — Grouping

```java id="k7m2q9"
ConcurrentHashMap<String, List<Integer>> map = new ConcurrentHashMap<>();

map.computeIfAbsent( "Java", k -> new ArrayList<>() ).add(10);
```

```
Conceptually:

```text id="v5x8r3"
Java → [10]
```

If Java already exists, the existing list is returned.

---

38. Important Concurrency Caveat

`computeIfAbsent()` makes the Map operation atomic, but that does **not automatically make the mutable value thread-safe**.

For example:

```java id="q3m7x9"
ConcurrentHashMap<String, List<Integer>>
```

```
doesn't mean the `ArrayList` stored as a value is thread-safe.

You still need to reason about:
```
```

```text id="n8x2m4"
Map concurrency
+
Value-object concurrency
```

separately.

This is a strong senior-level point.

---

39. `compute()`

You can atomically compute a new value:

```java id="r6k3p8" map.compute( key, (k, oldValue) -> ... );
```

This is useful for atomic per-key state transitions.

---

40. `merge()`

Another powerful operation:

```java id="x4m9q2"
map.merge(
    key,
    1,
    Integer::sum
);
```

This is useful for counters.

Conceptually:

```text id="j7p2k5" existing value + new value
```

with the remapping function applied atomically for the key.

```
# 41. Concurrent Counter Example

```java id="m8x3r7"
ConcurrentHashMap<String, Integer> counts =
    new ConcurrentHashMap<>();

counts.merge(
    "Java",
    1,
    Integer::sum
);
```

If:

```text id="f5q9m2" Java → 4
```

```
then after merge:

```text id="p3x7k8"
Java → 5
```

This is far safer than:

```java id="q6m2r4" counts.put( "Java", counts.get("Java") + 1 );
```

```
because the latter is a non-atomic read-modify-write sequence.

---

# 42. Why `get() + put()` Is Dangerous

Consider:

```java id="w8m4p2"
Integer value = map.get("Java");
map.put("Java", value + 1);
```

Two threads:

```text id="r7x3k9" Thread A reads 10 Thread B reads 10
```

A writes 11 B writes 11
```

````
Expected:

```text id="n5q8m1"
12
````

Actual:

```text id="c2p6r4" 11

````
Lost update.

Use:

```java id="j9m3x7"
merge()
````

or another suitable atomic approach.

---

43. Atomicity Is Usually Per-Key

Methods such as:

```text id="v6r2m8" putIfAbsent() compute() computeIfAbsent() computeIfPresent() merge() replace()

````
provide atomic semantics for their Map operation.

But don't infer:

> "Every sequence of multiple Map operations is automatically atomic."

For example:

```java id="x4k8m1"
if (map.containsKey(a)
        && map.containsKey(b)) {
    ...
}
````

is not one atomic transaction.

````

44. Compound Operations

This is a critical concurrency lesson.

Not atomic:

```java id="q7m2r9" if (!map.containsKey(k)) { map.put(k, v); }

```
Atomic alternative:

```java id="p3x8m5"
map.putIfAbsent(k, v);
```

Not atomic:

```java id="m6r2k7" map.put(k, map.get(k) + 1);

```
Better:

```java id="n8q4x1"
map.merge(k, 1, Integer::sum);
```

---

45. `replace()`

ConcurrentHashMap supports:

```java id="w5m9p2" map.replace(key, oldValue, newValue);

```
This is useful for compare-and-replace semantics.

Conceptually:

```text id="x3k7r6"
if current == oldValue
      ↓
replace with newValue
```

The operation is atomic for that mapping.

---

46. `replaceAll()`

You can also use:

```java id="j8p3m5" map.replaceAll( (key, value) -> ... );

```
But don't interpret this as an atomic transaction over the entire Map.

Concurrent collections provide strong guarantees for individual operations, not
arbitrary multi-operation business transactions.

---

47. `forEach()`

ConcurrentHashMap supports bulk traversal methods:

```java id="r5x2m8"
map.forEach(
    (key, value) -> ...
);
```

There are also parallel bulk-operation APIs in the concurrent collections framework.

The key point:

> These are designed with concurrent Map semantics in mind rather than requiring you to freeze the entire Map.

---

48. `mappingCount()`

ConcurrentHashMap provides:

```java id="p7m4x9" map.mappingCount();

```
which returns a `long`.

Why?

Because a very large concurrent map can conceptually exceed the `int` range used
by `Map.size()`.
```
```

````
For most applications:

```text id="q3r8k1"
size()
````

is sufficient.

But `mappingCount()` is worth knowing for interviews.

---

49. `isEmpty()`

You can use:

```java id="n6m2p8" map.isEmpty();

````
But remember:

> In a concurrently changing Map, the answer describes the state observed at
that point and can become stale immediately.

Concurrency means:

```text id="v4x9m3"
check
 ↓
another thread changes map
 ↓
previous result may no longer describe current state
````

---

50. ConcurrentHashMap Does Not Provide Snapshot Semantics

This is important.

Suppose another thread continuously changes the Map.

A call like:
````

```java id="j7r3m8" map.size();

```
doesn't mean:

> "Freeze the Map and calculate an immutable snapshot."

Likewise, iteration doesn't represent a guaranteed immutable snapshot.

---

51. No Ordering Guarantee

ConcurrentHashMap doesn't guarantee:

```text id="m4x8p2"
insertion order
```

or:

```text id="q7n3r9" sorted order

```
If you need insertion order:

```text id="c5m8x1"
LinkedHashMap
```

If you need sorted concurrent keys:

```text id="p2r7k4" ConcurrentSkipListMap

```
---

52. ConcurrentHashMap vs Hashtable

This is a very common interview comparison.

Hashtable

```text id="x8m3q5"
legacy
synchronized methods
```

```
coarser locking
no null key/value
```

ConcurrentHashMap

```text id="n6r2p8" modern fine-grained concurrency high throughput atomic compound operations no null key/value

```
---

53. Why Is Hashtable Usually Slower?

Consider:

```text id="m5x8r3"
Hashtable
   ↓
one synchronized method
   ↓
coarse-grained contention
```

ConcurrentHashMap:

```text id="q2p7k9" ConcurrentHashMap ↓ fine-grained coordination + CAS + volatile + localized synchronization

```
Therefore more threads can make progress concurrently.

---

54. ConcurrentHashMap vs `synchronizedMap()`

You can create:

```java id="r8m3x6"
Map<K,V> map =
    Collections.synchronizedMap(
        new HashMap<>()
    );
```

This provides synchronization around Map operations.

But it generally uses a common lock.

ConcurrentHashMap is designed specifically for concurrent access and offers:

```text id="v5q9m2" better concurrency + weakly consistent iteration + atomic concurrent Map methods
```

```
---

# 55. Comparison Table

| Feature | synchronizedMap | ConcurrentHashMap |
|---|---|---|
| Thread-safe | ✅ | ✅ |
| Fine-grained concurrency | Limited | ✅ |
| Concurrent reads | Serialized by wrapper locking semantics | Highly concurrent
|
| Null key | Depends on backing Map | ❌ |
| Null value | Depends on backing Map | ❌ |
| Weakly consistent iterator | ❌ | ✅ |
| Atomic `computeIfAbsent()` | Map default semantics depend on synchronization |
Designed for concurrent use |
| High-throughput concurrency | Lower | Better |

---

# 56. ConcurrentHashMap vs CopyOnWriteArrayList

These solve different problems.

```text id="j3m8q5"
ConcurrentHashMap
→ concurrent key-value access
```

```text id="x7r2p9" CopyOnWriteArrayList → concurrent list access with many reads/few writes
```

```
Choose based on the data structure required.

---

# 57. ConcurrentHashMap vs ConcurrentSkipListMap

This is an important senior-level comparison.

### ConcurrentHashMap

```text id="p5m9x3"
```

```
unordered
average O(1)
```

ConcurrentSkipListMap

```text id="k8r2m6" sorted O(log n)

```
If you need:

```text id="q4x7p1"
floorKey()
ceilingKey()
range queries
```

in a concurrent environment:

```text id="n6m3r8" ConcurrentSkipListMap

```
is often the appropriate choice.

---

58. ConcurrentHashMap and `computeIfAbsent()`

A common pattern:

```java id="w2p8m4"
ConcurrentHashMap<String, ExpensiveObject> cache =
    new ConcurrentHashMap<>();

ExpensiveObject value =
    cache.computeIfAbsent(
        key,
        k -> createExpensiveObject(k)
    );
```

This avoids the classic:

```text id="r7m3x9" check + put

```
race.

````
---

# 59. Important `computeIfAbsent()` Caveat

Don't perform complex Map mutations from inside the mapping function that
recursively modify the same Map.

For example, avoid designs where:

```java id="m8q2p5"
computeIfAbsent()
      ↓
mapping function
      ↓
same map.computeIfAbsent()
````

can recursively operate on the same key/map in an unsafe way.

Keep mapping functions:

```text id="x4r9m1" simple side-effect controlled non-recursive

````
where practical.

---

60. Mapping Function Should Not Return Null

For:

```java id="n6p3x8"
computeIfAbsent()
````

if the mapping function returns:

```text id="q5m8r2" null

````
the mapping isn't inserted.

This differs from ordinary `put()` because ConcurrentHashMap doesn't permit null
values.

````

```
# 61. `merge()` and Null

Since ConcurrentHashMap doesn't allow null values:

```text id="v8x2m4"
null
```

has special restrictions in remapping functions.

A remapping function returning null can remove an existing mapping according to the Map `merge()` contract.

This is a useful advanced detail.

---

62. `compute()` Returning Null

For:

```java id="j4m7q9" map.compute( key, (k, v) -> null );

```
if a mapping exists, returning null can remove it.

If no mapping exists, it remains absent.

This follows the `Map.compute()` semantics.

---

63. Atomic Counter Pattern

For concurrent counters:

```java id="p9x3m6"
ConcurrentHashMap<String, Long> counts =
    new ConcurrentHashMap<>();

counts.merge(
    "login",
    1L,
    Long::sum
);
```
```

This is much better than:

```java id="k2m8r5" counts.put( "login", counts.getOrDefault("login", 0L) + 1 );

```
because the latter isn't atomic.

---

64. Frequency Map

Classic pattern:

```java id="w6q4m2"
ConcurrentHashMap<String, Integer> frequency =
    new ConcurrentHashMap<>();

frequency.merge(
    word,
    1,
    Integer::sum
);
```

Useful for:

- Concurrent log processing
- Event counting
- Request counters
- Metrics aggregation

---

65. Cache Pattern

```java id="r3m8x1" ConcurrentHashMap<String, User> cache = new ConcurrentHashMap<>();

User user = cache.computeIfAbsent( userId, this::loadUser );

```
Conceptually:

```text id="n7p2k5"
lookup
  ↓
missing?
  ↓
```

```
load
  ↓
store
  ↓
return
```

The Map operation is designed to be safe under concurrent access.

---

66. Does ConcurrentHashMap Guarantee Only One Computation?

For `computeIfAbsent`, the mapping function is applied atomically for the key according to the method's contract.

The practical interview takeaway:

> Multiple threads don't simply perform an uncontrolled check-then-put race for the same missing key.

However, mapping functions should still be written carefully and should not assume arbitrary global serialization across unrelated keys.

---

67. Per-Key Atomicity

A useful mental model:

```text id="f5x9m3"
key A ↓ atomic operation

key B ↓ atomic operation
```

```
ConcurrentHashMap is designed to allow operations on unrelated keys to proceed
concurrently where possible.

This is one reason it scales well.

---

# 68. Does ConcurrentHashMap Make Mutable Values Thread-Safe?

No.
```

````
Example:

```java id="q8m2r6"
ConcurrentHashMap<String, ArrayList<Integer>>
````

The Map can safely manage mappings concurrently, but:

```text id="p4x7n1" ArrayList
````

````
is still not thread-safe.

You need to synchronize or use a concurrent collection for the value if multiple
threads mutate it.

---

# 69. Example

Unsafe assumption:

```java id="j3m8x5"
map.computeIfAbsent(
    "A",
    k -> new ArrayList<>()
).add(10);
````

The Map operation may be thread-safe, but concurrent `.add()` calls on the same `ArrayList` are not automatically made safe by the Map.

Possible alternatives include:

```text id="m7q2r9" CopyOnWriteArrayList
````

````
or:

```text id="x4p8k1"
synchronizedList
````

depending on workload.

````

70. Hash Distribution Still Matters

ConcurrentHashMap is hash-based.

Therefore poor `hashCode()` implementations can still cause:

```text id="v6m3q8" collisions

```
and increased contention/work in a bin.

Good hash distribution remains important.

---

71. Hash Collision ≠ Thread Collision

These are different concepts.

Hash collision

Two keys map to the same bin:

```text id="n2r7x5"
A → bin 4
B → bin 4
```

Thread contention

Multiple threads compete to modify the same shared state.

They can be related:

```text id="p8m4q1" many keys ↓ same bin ↓ more localized contention

```
but they are not the same thing.

---

72. Why HashMap Is Not Thread-Safe

A simplistic answer:

> "Because multiple threads can modify it."
```
```

````
Better answer:

> HashMap doesn't provide the synchronization and memory-visibility guarantees
required for concurrent mutation. Concurrent operations can race, producing lost
updates and inconsistent observations, and compound operations such as check-
then-act are not atomic.

---

# 73. Does ConcurrentHashMap Make Every Operation Atomic?

No.

Individual supported Map operations have defined concurrent semantics, and many
compound operations such as:

```text id="m9x3r7"
putIfAbsent
compute
computeIfAbsent
merge
replace
````

are atomic at the Map-operation level.

But arbitrary sequences remain non-atomic.

Example:

```java id="q7p2m5" if (map.containsKey(a)) { map.remove(b); }

````
is not one atomic transaction.

---

74. `putIfAbsent()` vs `computeIfAbsent()`

`putIfAbsent`

You already have the value:

```java id="f3x8m2"
map.putIfAbsent(
    key,
````
````

```
    value
);
```

`computeIfAbsent`

The value should be generated lazily:

```java id="k5m9r1" map.computeIfAbsent( key, k -> createValue(k) );
```

```
Think:

```text id="n8q2x6"
Value already available?
      ↓
putIfAbsent()

Need to create lazily?
      ↓
computeIfAbsent()
```

---

75. `replace()` vs `put()`

```java id="r4m8x2" map.put(key, newValue);
```

```
unconditionally inserts/replaces.

But:

```java id="j7p3n9"
map.replace(
    key,
    oldValue,
    newValue
);
```

only replaces if the current value matches the expected old value.

Useful for atomic conditional updates.

---

76. Compare-and-Set Mental Model

Think:

```text id="x5m2q8" Current value == expected? ↓ YES ↓ replace

```
This is similar conceptually to CAS-style coordination.

It is useful for implementing simple state transitions.

---

77. Example — State Transition

Suppose:

```text id="m8r3p5"
order → NEW
```

You want:

```text id="q6x9k2" NEW → PROCESSING

```
You can use:

```java id="v4m7x1"
map.replace(
    orderId,
    "NEW",
    "PROCESSING"
);
```

The operation succeeds only if the expected current state is still `"NEW"`.

---

78. Weakly Consistent Views

Methods such as:

```java id="n3x8m5" keySet() values() entrySet()
```

````
provide views designed for concurrent use.

Iteration over them is weakly consistent.

This means:

```text id="p7r2k9"
safe under concurrent modification
+
not snapshot
+
may reflect some concurrent changes
````

---

79. Fail-Fast vs Weakly Consistent

Memorize:

```text id="j4m8x2"
ArrayList
HashMap
↓
fail-fast iterator
````

````
versus:

```text id="q9x3r6"
ConcurrentHashMap
   ↓
weakly consistent iterator
````

And:

```text id="m5k7p1"
CopyOnWriteArrayList
↓
snapshot-style iterator
````

````
These are three different concepts.

---

# 80. Snapshot vs Weak Consistency

### Snapshot-style

Iterator sees a stable snapshot:

```text id="r8x2m5"
````
````

```
Start
 ↓
snapshot
 ↓
iterate snapshot
```

Weakly consistent

Iterator sees the collection while it may change:

```text id="v3m7q9"
Start ↓ iterate ↘ changes may occur
```

```
ConcurrentHashMap uses the second model.

---

# 81. Is ConcurrentHashMap "Fail-Safe"?

Interview terminology:

You may hear:

```text id="k2m8x4"
fail-safe
```

for concurrent iterators.

But a stronger answer is:

> ConcurrentHashMap provides **weakly consistent iterators**, which do not throw `ConcurrentModificationException` merely because concurrent modifications occur.

Use the JDK terminology when possible.

---

82. ConcurrentHashMap and `ConcurrentModificationException`

Consider:

```java id="n6r3p8"
for (String key : map.keySet()) {
    map.put("X", 10);
}
```
```

```
With ConcurrentHashMap, this doesn't inherently produce the fail-fast behavior
associated with HashMap iterators.

However, modifying a collection while traversing it can still have application-
level consequences, so don't treat weak consistency as "iteration sees
everything predictably."

---

# 83. Performance Characteristics

Typical average complexity:

```text id="x5m8q2"
get()    → O(1)
put()    → O(1)
remove() → O(1)
```

subject to hashing, collisions, resizing, contention, and implementation details.

Tree bins can make heavily collided lookup behavior logarithmic within the tree structure.

---

84. Is ConcurrentHashMap Faster Than HashMap?

Not necessarily for single-threaded workloads.

HashMap has less concurrency-related machinery.

The advantage of ConcurrentHashMap appears when:

```text id="p3r7m9" multiple threads + shared Map
```
are involved.

Don't say:

> "ConcurrentHashMap is always faster."

Better:

> "It is designed to provide much better concurrency and scalability than
coarse-grained synchronization, while retaining efficient average Map
```

````
operations."

---

# 85. Is `ConcurrentHashMap` Lock-Free?

**No.**

Another common interview trap.

It uses:

```text id="j8m4q2"
CAS
+
volatile
+
synchronization
````

Some operations may proceed without locking, but the overall data structure is not simply "lock-free."

---

86. Is ConcurrentHashMap Segmented in Java 8?

The old segment architecture from Java 7 was removed.

Modern implementations use a table/bin-oriented design rather than the old fixed Segment array.

If asked:

> "Does ConcurrentHashMap use segments?"

Answer:

> Java 7 used segmentation; Java 8+ removed the Segment-based architecture and uses finer-grained bin-level coordination with CAS and synchronization.

Excellent interview answer.

---

87. Treeification

Like HashMap, collision-heavy bins can transition to tree-based structures.
````

Conceptually:

```text id="m3x7q1"
bin ↓
Node → Node → Node → ...
↓ too many collisions
TreeBin ↓
Red-Black Tree
```

```
This helps protect performance under poor hash distributions.

---

# 88. Resize Process

When enough entries are added:

```text id="p8m2r6"
size/load threshold
       ↓
resize
       ↓
larger table
```

Modern ConcurrentHashMap can allow threads encountering the resize to help transfer bins.

Think:

```text id="x5q9m3"
resize ↓
multiple threads can help
```

```
rather than one thread necessarily doing all migration work alone.

---

# 89. Why Resizing Is Complicated

Because other threads may simultaneously perform:

```text id="k7m3x8"
get()
put()
remove()
```

while buckets are being moved.

Therefore the implementation uses specialized coordination/state mechanisms to ensure threads can safely interact with the old and new tables during transfer.

For interviews:

Understand **cooperative resizing**, not every source-level transfer detail.

---

90. `LongAdder` and ConcurrentHashMap

This is a popular high-performance counter pattern.

Instead of:

```java id="r4m8q2" ConcurrentHashMap<String, Long>
```

```
you can use:

```java id="n6x3m7"
ConcurrentHashMap<String, LongAdder>
```

Example:

```java id="p8q2k5" ConcurrentHashMap<String, LongAdder> counts = new ConcurrentHashMap<>();

counts.computeIfAbsent( "requests", k -> new LongAdder() ).increment();
```

```
This can reduce contention for heavily updated counters.

---

# 91. Why LongAdder?

With many threads incrementing the same counter:

```text id="m7x4r1"
AtomicLong
    ↓
one shared counter
    ↓
high contention
```

`LongAdder` can spread updates across internal cells and combine them when reading.

This is especially useful for high-contention metrics.

```
```

92. ConcurrentHashMap + LongAdder Pattern

This is worth remembering:

```java id="q5m8x2" ConcurrentHashMap<String, LongAdder> counters = new ConcurrentHashMap<>();

counters.computeIfAbsent( key, k -> new LongAdder() ).increment();

```
Conceptually:

```text id="j3r7m9"
key
 ↓
LongAdder
 ↓
concurrent increments
```

Excellent production-level pattern.

---

93. Cache Design

For a basic concurrent cache:

```java id="x8m2q5" ConcurrentHashMap<String, User> cache = new ConcurrentHashMap<>();

User user = cache.computeIfAbsent( userId, this::loadUser );

```
Advantages:

```text id="p6r3k9"
thread-safe Map
+
atomic per-key initialization
+
no explicit global lock
```

But production caches often also need:

```text id="v4m8x2" TTL maximum size eviction refresh metrics
```

````
ConcurrentHashMap alone doesn't provide those policies.

---

# 94. Common Interview Trap

### Question:

> Is `ConcurrentHashMap<String, List<String>>` fully thread-safe?

Answer:

**No.**

The Map's mapping operations are concurrent-safe, but the `List` values may not be.

You must separately make the values safe if multiple threads mutate the same list.

---

# 95. Common Interview Trap

### Question:

> Can I store `null` in ConcurrentHashMap?

No.

Neither:

```java id="k5x9r3"
put(null, value)
````

nor:

```java id="q7m2p8"
put(key, null)
```

````
is allowed.

---

# 96. Common Interview Trap
````
````

```
### Question:

> Does ConcurrentHashMap lock the entire Map during `put()`?

No.

Modern implementations use fine-grained coordination rather than one global lock
for every update.

---

# 97. Common Interview Trap

### Question:

> Is ConcurrentHashMap completely lock-free?

No.

It uses a combination of:

```text id="n8r4m2"
CAS
volatile operations
synchronization
```

depending on the operation and state of the bin/table.

---

98. Common Interview Trap

Question:

> Is its iterator fail-safe?

Prefer:

> **Weakly consistent.**

It doesn't throw `ConcurrentModificationException` merely because concurrent modifications happen, and it does not provide snapshot semantics.

```

99. Common Interview Trap

Question:

> Why not synchronize HashMap manually?

You can, but:

```text id="x6m3q9" synchronized HashMap ↓ coarse-grained locking ↓ reduced concurrency

```
ConcurrentHashMap is designed specifically to provide better concurrent
scalability.

---

100. Common Interview Trap

Question:

> Why not use Hashtable?

Because Hashtable is a legacy synchronized Map with coarser synchronization
semantics.

For modern concurrent Map requirements:

```text id="p8r2m5"
ConcurrentHashMap
```

is generally preferred.

---

101. Common Interview Trap

Question:

> Is `get()` atomic?

A single `get()` has well-defined concurrent behavior and safely observes the Map's state according to ConcurrentHashMap's memory-consistency guarantees.

But:

```java id="j3x7m1" get() + get() + put()

```
as a larger sequence is not automatically atomic.

---

102. Common Interview Trap

Question:

```java id="m8q4r2"
if (!map.containsKey(k)) {
    map.put(k, v);
}
```

Is this thread-safe?

No.

Use:

```java id="v5x2p8" map.putIfAbsent(k, v);

```
when that is the required semantics.

---

103. Common Interview Trap

Question:

```java id="q7m3x9"
map.put(
    key,
    map.get(key) + 1
);
```

Is this thread-safe?

No.

Use:

```java id="r4k8m2" map.merge( key, 1, Integer::sum );


```

```
or an appropriate `LongAdder` pattern for high-contention counters.

---

# 104. ⭐ Coding Question

What happens?

```java id="x3m8q5"
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();

map.put("A", 10);
map.putIfAbsent("A", 20);

System.out.println(map.get("A"));
```

Output:

```text id="j7r2p9" 10
```

```
Because `A` already exists.

---

# 105. ⭐ Coding Question

```java id="p6m3x8"
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();

map.put("A", 10);

map.merge(
    "A",
    5,
    Integer::sum
);
```

Result:

```text id="q8r4m1" A → 15
```

```
---

# 106. ⭐ Coding Question

```java id="n5x9k2"
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();

map.computeIfAbsent(
    "A",
    k -> 100
);
```

Result:

```text id="m7p3r8" A → 100

```
If A already exists, the mapping function isn't used to replace the existing
mapping.

---

107. ⭐ Coding Question

```java id="r4m8q2"
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();

map.put("A", 10);

map.compute(
    "A",
    (k, v) -> v + 5
);
```

Result:

```text id="x3k7p9" A → 15

```
---

108. ⭐ Coding Question
```

```java id="m6q2x8"
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();

map.put("A", 10);

map.replace(
    "A",
    10,
    20
);
```

Result:

```text id="p5r8m3"
A → 20
```

```
If another thread changes A before the conditional replacement, the replacement
may fail.

---

# 109. ⭐ Coding Question

What happens?

```java id="k8m4x2"
ConcurrentHashMap<String, String> map =
    new ConcurrentHashMap<>();

map.put(null, "A");
```

Answer:

```text id="q3p7m9"
NullPointerException
```

```
---

# 110. ⭐ Coding Question

What happens?

```java id="x5r2m8"
ConcurrentHashMap<String, String> map =
    new ConcurrentHashMap<>();
```

```
map.put("A", null);
```

Answer:

```text id="j7m3q6" NullPointerException

---

# 111. ⭐ Coding Question – Race Condition

Two threads execute:

```java id="p4x8m1"
if (!map.containsKey("A")) {
    map.put("A", 100);
}
```

Could both threads believe A is absent?

Yes.

Because:

```text id="r7m2q9" Thread A → containsKey → false Thread B → containsKey → false

Then both can call `put()`.

Use:

```java id="n5k8x3"
map.putIfAbsent("A", 100);
```

---

112. ⭐ Coding Question — Counter

Two threads execute:

```java id="q8m3r5" map.put( "count", map.get("count") + 1 );
```

```
Is this atomic?

**No.**

Possible lost update.

Better:

```java id="x4m7p2"
map.merge(
    "count",
    1,
    Integer::sum
);
```

---

113. ⭐ Coding Question — Iteration

Thread A:

```java id="j6r2m9" for (Integer key : map.keySet()) { System.out.println(key); }
```

Thread B:

```java id="p8m4x1"
map.put(100, "X");
```

Does ConcurrentHashMap necessarily throw:

```text id="v3q7m5" ConcurrentModificationException
```

No.

Its iterators are weakly consistent.

---

114. ⭐ Senior Interview Question

Explain ConcurrentHashMap architecture in Java 8+.

Strong answer:
```

> Java 8+ ConcurrentHashMap no longer uses the old Segment-based architecture from Java 7. It uses a table of bins, CAS for certain initialization/update paths, volatile memory semantics for visibility, and synchronized blocks on specific bins when required. Collision-heavy bins can use tree structures. This provides much finer-grained concurrency than locking the entire Map.

---

115. ⭐ Senior Interview Question

Why is ConcurrentHashMap better than synchronizedMap for high concurrency?

Answer:

> A synchronized Map typically serializes operations through a common lock, whereas ConcurrentHashMap uses a more fine-grained concurrency design. Reads can proceed concurrently, and updates generally coordinate only around the relevant part of the table, allowing unrelated operations to make progress concurrently.

---

116. ⭐ Senior Interview Question

Why doesn't ConcurrentHashMap allow null?

Answer:

> Null would make the result of operations such as `get()` ambiguous because `null` could mean either "no mapping" or "mapping exists with a null value." ConcurrentHashMap's atomic and concurrent APIs avoid this ambiguity by prohibiting null keys and values.

---

117. ⭐ Senior Interview Question

What is the difference between fail-fast and weakly consistent iterators?

Answer:

> A fail-fast iterator detects certain structural modifications and may throw `ConcurrentModificationException`. A weakly consistent iterator, as used by ConcurrentHashMap, can iterate while concurrent modifications occur without failing merely because of those modifications. It may reflect some concurrent changes and doesn't represent a guaranteed snapshot.

````
---

# 118. ⭐ Senior Interview Question

### Why is `containsKey()` + `put()` unsafe?

Because it is a:

```text id="c7m2x9"
check-then-act
````

sequence.

The state can change between the two operations.

Use:

```java id="r4p8n1" putIfAbsent()
````

````
for the atomic operation.

---

# 119. ⭐ Senior Interview Question

### Why is `get()` + `put()` unsafe for counters?

Because:

```text id="m5x3q8"
read
+
modify
+
write
````

is not one atomic operation.

Use:

```java id="q7r2m9" merge()
````

````
or a suitable counter structure.
````

```
---

# 120. ⭐ Senior Interview Question

### Does ConcurrentHashMap guarantee snapshot iteration?

No.

Its iterators are weakly consistent, not snapshot iterators.

---

# 121. ⭐ Senior Interview Question

### Does ConcurrentHashMap guarantee ordering?

No.

It doesn't guarantee insertion order or sorted order.

Use:

```text id="p6m8x2"
ConcurrentSkipListMap
```

when concurrent sorted-key behavior is required.

---

122. ⭐ Senior Interview Question

Can ConcurrentHashMap make its values thread-safe?

No.

If the value is:

```java id="x3m7q9" ArrayList
```

```
the list is still an ArrayList.

Map thread safety and value-object thread safety are separate concerns.

---
```
```

```
# 123. ⭐ Senior Interview Question

### What is CAS?

Answer:

> CAS, or Compare-And-Swap, atomically checks whether a memory location still
contains an expected value and, if so, replaces it with a new value.
ConcurrentHashMap uses CAS for certain low-level concurrent update paths.

---

# 124. ⭐ Senior Interview Question

### What happens during heavy hash collisions?

Conceptually:

```text id="j8m4p2"
many keys
    ↓
same bin
    ↓
collision chain
    ↓
treeification when conditions are met
    ↓
TreeBin / Red-Black Tree
```

This helps maintain better lookup behavior under collision-heavy conditions.

---

125. ⭐ Senior Interview Question

What changed between Java 7 and Java 8?

Memorize:

```text id="q3r7m8"
Java 7 → Segments

Java 8+ → CAS → bin-level synchronization → volatile operations → tree bins → cooperative resizing
```

```
This is one of the most frequently asked ConcurrentHashMap questions.
```
```

```
---

# 126. ConcurrentHashMap Decision Framework

```text id="x5m2r8"
Need Map
    ↓
Multiple threads access/update it?
    ↓
   NO ──────────→ HashMap
    |
   YES
    ↓
Need sorted keys?
    ↓
 YES ──────────→ ConcurrentSkipListMap
    |
   NO
    ↓
ConcurrentHashMap
```

Need insertion/access order?

```text id="m7p3q9" ConcurrentHashMap doesn't provide it.

```
You need a different concurrency design.

---

127. Complete Map Selection

Now the Map landscape should look like:

```text id="r8m4x2"
                              Map
                               |
       ┌───────────────────────┼──────────────────────┐
       ↓                       ↓                      ↓
    HashMap             LinkedHashMap             TreeMap
       ↓                       ↓                      ↓
  General fast          insertion/access        sorted keys
    lookup                  order
```

Specialized:
```

```text id="p5x8m3" WeakHashMap → weak keys

IdentityHashMap → identity keys

EnumMap → enum keys

ConcurrentHashMap → concurrent unordered Map

ConcurrentSkipListMap → concurrent sorted Map

---

# 128. 🧠 Final Mental Model

Remember ConcurrentHashMap as:

```text id="j4m8q2"
                ConcurrentHashMap
                        |
          ┌─────────────┼─────────────┐
          ↓             ↓             ↓
        CAS         Volatile      Synchronization
          |             |             |
          └─────────────┼─────────────┘
                        ↓
              Fine-grained concurrency
                        ↓
                High throughput
```

And:

```text id="x7p3m9" HashMap → not thread-safe

synchronizedMap → thread-safe → coarse-grained locking

ConcurrentHashMap → thread-safe → fine-grained concurrency → atomic Map operations → weakly consistent iterators ```

---

# 🔥 Chapter Summary

| Concept | Key Point |
|---|---|
| ConcurrentHashMap | Thread-safe concurrent Map |
| Package | `java.util.concurrent` |
| Java 7 | Segment-based architecture |
| Java 8+ | CAS + bin-level synchronization + volatile |
| Entire Map lock | ❌ |
| Concurrent reads | ✅ |
| Concurrent updates | ✅ |
| Average `get()` | O(1) |
| Average `put()` | O(1) |
| Null key | ❌ |
| Null value | ❌ |
| Ordering | None guaranteed |
| Tree bins | ✅ |
| CAS | Used in concurrent update paths |
| Resizing | Cooperative/concurrent |
| Iterator | Weakly consistent |
| Fail-fast | ❌ |
| Snapshot iterator | ❌ |
| `putIfAbsent()` | Atomic conditional insertion |
| `computeIfAbsent()` | Atomic lazy computation |
| `compute()` | Atomic per-key computation |
| `merge()` | Atomic per-key merge |
| `replace()` | Conditional replacement |
| Thread-safe values | ❌ not automatically |
| High-contention counters | `LongAdder` can help |
| Concurrent sorted Map | `ConcurrentSkipListMap` |

# 🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] Why HashMap isn't thread-safe
- [ ] Why ConcurrentHashMap exists
- [ ] Java 7 Segment architecture
- [ ] Java 8+ architecture
- [ ] CAS
- [ ] Volatile semantics
- [ ] Bin-level synchronization
- [ ] Why the whole Map isn't locked
- [ ] Concurrent reads
- [ ] Concurrent writes
- [ ] Hash collisions
- [ ] Tree bins
- [ ] Treeification
- [ ] Resizing
- [ ] Cooperative resizing
- [ ] Why null keys/values aren't allowed
- [ ] `putIfAbsent()`
- [ ] `computeIfAbsent()`
- [ ] `compute()`
- [ ] `merge()`
- [ ] `replace()`
- [ ] Atomic vs non-atomic compound operations
- [ ] Weakly consistent iterators
- [ ] Fail-fast vs weakly consistent
- [ ] ConcurrentHashMap vs HashMap
- [ ] ConcurrentHashMap vs Hashtable
- [ ] ConcurrentHashMap vs synchronizedMap
- [ ] ConcurrentHashMap vs ConcurrentSkipListMap
- [ ] ConcurrentHashMap + LongAdder
- [ ] Why thread-safe Map doesn't make mutable values thread-safe
- [ ] Real-world cache/counter patterns

# 🏆 Map Interview Master Table

| Map | Ordering | Thread-Safe | Key Semantics | Internal Idea | Best Use |
|---|---|---|---|---|---|
| `HashMap` | None guaranteed | ❌ | `equals()` / `hashCode()` | Hash table | General lookup |

| Map | Ordering | Thread-Safe | Key Semantics | Internal Idea | Best Use |
|---|---|---|---|---|---|
| `LinkedHashMap` | Insertion/ access | ❌ | `equals()` / `hashCode()` | Hash table + links | Ordered Map / LRU |
| `TreeMap` | Sorted | ❌ | `Comparable` / `Comparator` | Red-Black Tree | Sorted/ range operations |
| `WeakHashMap` | None guaranteed | ❌ | Weak keys | Weak references | Lifecycle-sensitive metadata |
| `IdentityHashMap` | None guaranteed | ❌ | `==` | Array + probing | Object identity |
| `EnumMap` | Enum declaration order | ❌ | Enum keys | Array-based | Enum → value |
| `ConcurrentHashMap` | None guaranteed | ✅ | `equals()` / `hashCode()` | Concurrent hash table | High-concurrency Map |
| `ConcurrentSkipListMap` | Sorted | ✅ | Ordering | Skip list | Concurrent sorted Map |

# 🚀 Next Chapter

# Chapter 22 — Map Deep Dive: `HashMap` Internals + Collisions + Rehashing + Treeification

This is the **most important Map interview chapter**.

We'll go deeper into:

- HashMap bucket calculation
- `hash()` spreading
- `hashCode()` vs `equals()`
- Bucket indexing
- Node structure
- Collision mechanics
- Linked-list bins
- Tree bins

• Red-Black Tree conversion
• `TREEIFY_THRESHOLD`
• `UNTREEIFY_THRESHOLD`
• `MIN_TREEIFY_CAPACITY`
• Load factor
• Capacity
• Threshold
• Rehashing
• Resizing
• Power-of-two capacity
• Why `n - 1 & hash` is used
• Java 7 vs Java 8 HashMap
• Why HashMap can become corrupted under concurrent writes
• Mutable-key problem
• `equals()` / `hashCode()` contract
• Worst-case vs average complexity
• 30+ senior-level interview questions